

Reflective Report - Accessible Games Application

1) Architecture and Design

The C4 architecture diagrams underwent three iterations throughout the project, with version history maintained in our GitLab Wiki (see Figures 1-3). Architecture documents should ideally be completed as a group before development begins. We created basic documentation of our initial idea; however, our diagrams required constant revision as the project evolved. Within a condensed three-week timeline, requirements moved around at various points each week, and technical decisions changed, aligning with the iterative nature of agile development (Fowler, 2017).

The first iteration reflected initial assumptions about system scope and ORM choices. Treating diagrams as living documents with three iterations provided historical context that static documentation cannot offer.

Context (Latest)

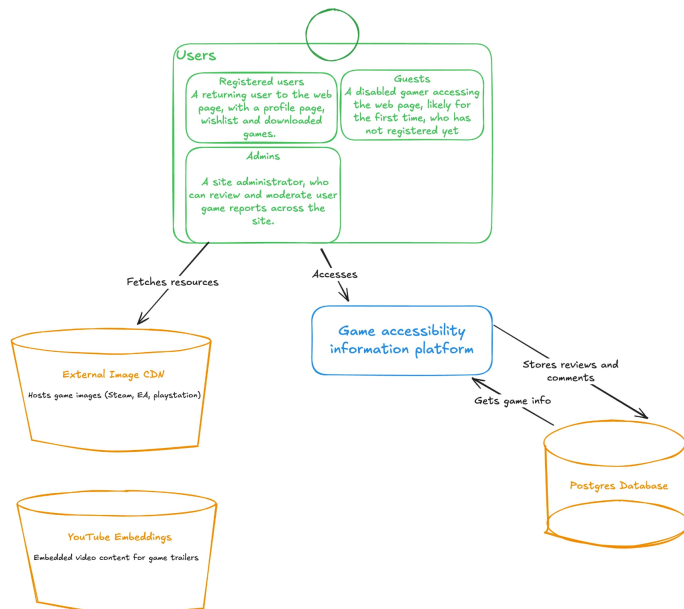


Figure 1: C4 Context Diagram

Container (Latest)

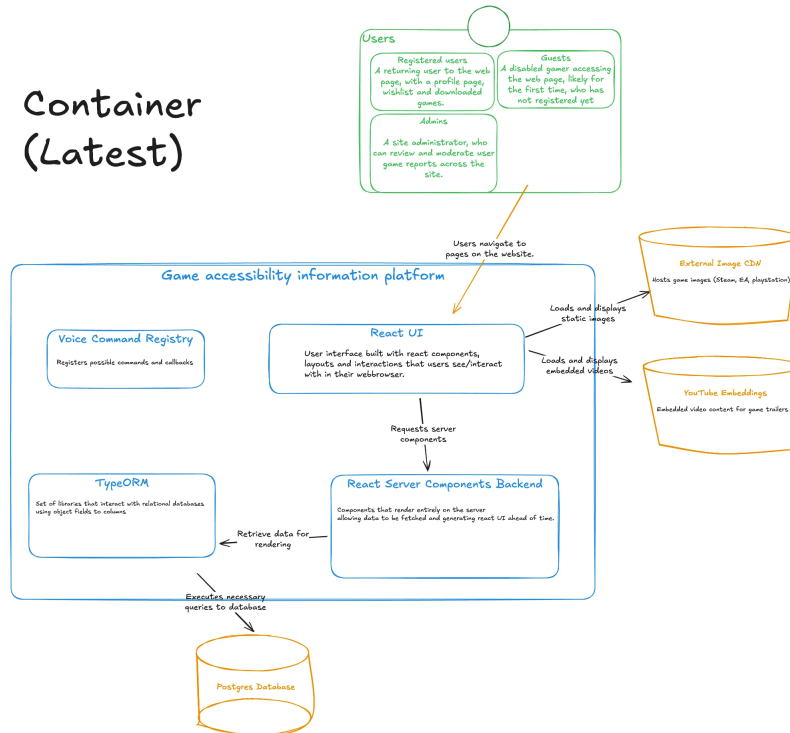


Figure 2: C4 Container Diagram

Components (Latest)

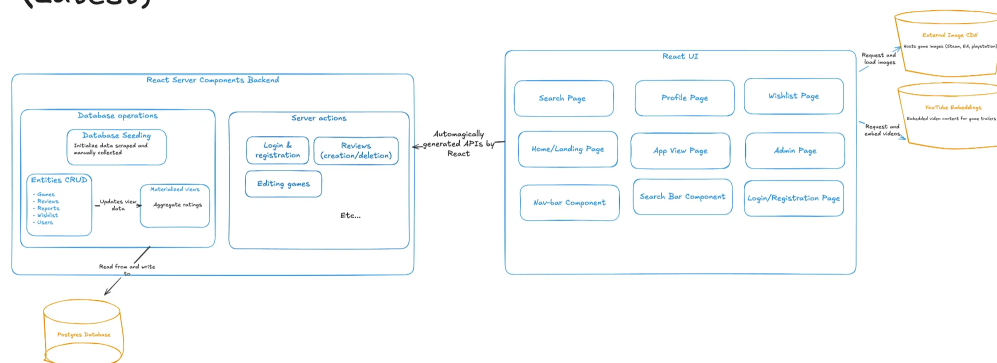


Figure 3: C4 Component Diagram

The first week focused heavily on documentation and setting up the repository without establishing clear component boundaries, resulting in unnecessary team efforts when the scope then changed. For future university projects, I would opt for an MVP-first approach: build a minimal working prototype in week one to validate technical feasibility and establish boundaries that arise once we have basic implementation, then document the architecture that emerges. This opposes the traditional approach but suits rapid, exploratory development where design choices will change as development issues occur. The documentation we created initially became obsolete within the first week; had we prototyped first, our architecture diagrams would have reflected actual system behaviour rather than theoretical design. For commercial projects with longer timelines and clearer requirements, greater upfront architectural investment makes sense - the cost of changing architecture mid-project scales badly with team size. If I were working

as a developer, I would schedule architecture review sessions twice a sprint (i.e., twice a week), ensuring diagrams remain current and serve as onboarding resources for potential new team members. The C4 Model emphasises creating developer-friendly diagrams that aid understanding (Brown, 2018) - their value lies in shared understanding rather than prescriptive accuracy, making them well-suited to agile environments where architecture evolves alongside implementation.

2) Frameworks and External Libraries

Our stack comprised React Server Components with Vite, TypeORM for persistence, and Bun as the JavaScript runtime. A knowledge gap emerged around RSC file conventions - I had not appreciated the distinction between `.ts` and `.tsx` extensions (React, 2024). This confused me when server-side functions failed to compile. The server/client boundary introduces new mental models requiring dedicated study (Vercel, 2024). In a software developer career, I would allocate adequate time for technological architecture and framework research before beginning development to avoid learning things later that I should have been aware of at the time.

On December 3rd, 2025, hours after our group submission, a critical vulnerability (CVE-2025-55182) was disclosed in React Server Components, rated CVSS 10.0, allowing unauthenticated remote code execution (React, 2025). Security researchers demonstrated near-100% exploitation reliability (Wiz, 2025), with Microsoft reporting active exploitation attempts from December 5th (Microsoft, 2025). This post-submission disclosure highlighted the inherent security risks of adopting newly released frameworks - React Server Components had been in production for only weeks before this critical flaw was uncovered. If I were working professionally, I would maintain daily monitoring of the framework's security advisories and keep up with the patch deployment process to address vulnerabilities as they emerge. While our team maintained awareness of security patches throughout development and monitored the React ecosystem closely, this incident reinforces that even thorough security practices cannot fully mitigate the risks of early framework adoption in production environments.

Bun provided performance improvements over Node.js (Bun, 2025). However, I made a critical oversight: the initial CI pipeline executed `bun install` and `bun run test` but lacked `bun run build`. Compilation errors went undetected until team members had local build issues. This taught me that successful delivery depends on properly automating all validation stages (Humble and Farley, 2010). In a professional CI/CD environment, I'd push for pipeline templates enforcing mandatory build, test, and deployment stages. As a team, we should have noticed this critical pipeline stage was missing, but we did well to catch it as early as we did.

TypeORM's Active Record pattern accelerated the development of my Report entity and admin panel CRUD operations (see Figures 4-5). Extending `BaseEntity` provided inherited methods like `save()` and `remove()`, with readable decorator syntax (TypeORM, 2024). However, ORM-generated queries can be naive and inefficient (Neurelo, 2024) - acceptable here but problematic for performance-critical commercial

applications. For future projects, I would use TypeORM's Query Builder earlier. In an industry setting, I would establish query performance benchmarks early on.

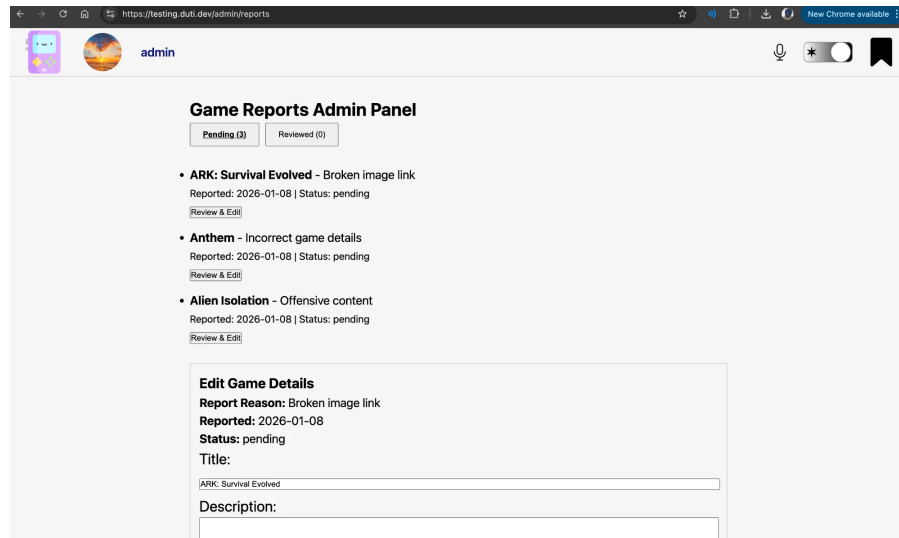


Figure 4: Admin Panel - Game Reports interface with CRUD operations

Edit Game Details
Report Reason: Broken image link
Reported: 2026-01-08
Status: pending
Title:

Description:

Image URL:

 ARK: Survival Evolved

Figure 5: Edit Game Details form with status updates and deletion

3) Documentation

Documenting Bun installation, PostgreSQL configuration, and TypeORM's automatic migration behaviour made me realise I understood things I'd never actually written down. Clarifying that `bun run dev` starts Vite's development server on port 5173 seemed obvious, but was essential to document for onboarding developers who are unfamiliar with the stack. This aligns with Rüping's (2003) principle that documentation should be lean and based on what actually needs to be known. In a professional environment, I would ensure that documentation of all processes is thorough, as aspects that may seem obvious to our team and me would not be obvious to others trying to run our application.

The CI pipeline's evolution demonstrated the iterative nature of testing. The initial configuration, missing the build step, meant the team operated with false confidence. For this project, adding build verification immediately improved merge request quality. For future projects, I would establish and ask for code reviews of builds in CI pipelines before advanced feature development begins.

During code review, a teammate identified that deleting a game would break associated report entities due to foreign key references. The @ManyToMany relationship meant orphaned reports would cause database integrity errors. I resolved this by ensuring that `deleteGame` removes associated reports first, maintaining referential integrity. I lacked proper integration and unit tests for my features. For future projects, I would implement test-driven development for complex features. In a commercial team, I'd push for mandatory integration tests covering database cascading operations as part of the definition of done. In simpler terms, just writing tests alongside developing a feature and not leaving it until last as excuses can be made; without testing in development, the scope of my feature is very short-sighted.

The first week's focus on documentation without parallel development created an imbalance. By the second week we found our rhythm and development and documentation became fluid. Future projects should interleave documentation with development incrementally. In a professional agile team, this would mean adding documentation tasks to each sprint - following the agile principle of working software over documentation, while still keeping proper version control and records.

Bibliography

Bass, L., Clements, P. and Kazman, R. (2012) *Software Architecture in Practice*. 3rd edn. Upper Saddle River, NJ: Addison-Wesley Professional.

Brown, S. (2018) 'The C4 model for visualising software architecture'. Available at: <https://c4model.com/> (Accessed: 4 December 2025).

Bun (2025) *Bun - A Fast All-in-One JavaScript Runtime*. Available at: <https://bun.sh/> (Accessed: 4 December 2025).

Fowler, M. (2017) *Foreword to Building Evolutionary Architectures*. Available at: <https://martinfowler.com/articles/evo-arch-forward.html> (Accessed: 4 December 2025).

Humble, J. and Farley, D. (2010) *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Upper Saddle River, NJ: Addison-Wesley Professional.

Microsoft (2025) *Defending against the CVE-2025-55182 vulnerability in React Server Components*. Available at: <https://www.microsoft.com/en-us/security/blog/2025/12/15/defending-against-the-cve-2025-55182-react2shell-vulnerability-in-react-server-components/> (Accessed: 8 January 2026).

Neurelo (2024) *What is an ORM and When Developers Should and Shouldn't Use It*. Available at: <https://dev.to/neurelo/what-is-an-orm-and-when-developers-should-and-shouldnt-use-it-1hm2> (Accessed: 4 December 2025).

React (2024) *Using TypeScript*. Available at: <https://react.dev/learn/typescript> (Accessed: 4 December 2025).

React (2025) *Critical Security Vulnerability in React Server Components*. Available at: <https://react.dev/blog/2025/12/03/critical-security-vulnerability-in-react-server-components> (Accessed: 8 January 2026).

Rüping, A. (2003) *Agile Documentation: A Pattern Guide to Producing Lightweight Documents for Software Projects*. Chichester: John Wiley & Sons.

TypeORM (2024) *Many-to-one / one-to-many relations*. Available at: <https://typeorm.io/many-to-one-one-to-many-relations> (Accessed: 8 January 2026).

Vercel (2024) *React Server Components*. Available at: <https://nextjs.org/docs/app/building-your-application/rendering/server-components> (Accessed: 8 January 2026).

Wiz (2025) *React2Shell (CVE-2025-55182): Critical React Vulnerability*. Available at: <https://www.wiz.io/blog/critical-vulnerability-in-react-cve-2025-55182> (Accessed: 8 January 2026).